Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

Kết luận **2**
Tổng quan quá trình 2
Lộ trình chuyển đổi thực tế 3
Khi nào áp dụng — heuristic thực tế 3
Điều không thay đổi 4

Kết luận

> "Architecture is not a destination, it's a continuous journey of deliberate decisions and conscious trade-offs."
> — tổng hợp từ Richardson [2a] và Newman [4a]

Sau 12 chương, chúng ta đã đi qua toàn bộ quá trình kỹ thuật của KBM LMS — từ monolith ban đầu đến kiến trúc microservices đang tiếp tục hoàn thiện. Chương kết này không giới thiệu pattern mới, mà tổng hợp các quyết định đã học thành một **lộ trình thực tế có thứ tự ưu tiên**.

Tổng quan quá trình

Nhìn lại, KBM đã áp dụng các nguyên lý của kiến trúc hướng dịch vụ (SOA) như tính tự chủ (Autonomy) và hợp đồng dịch vụ (Service Contract), nhưng khéo léo tránh được cái bẫy của SOA truyền thống. Thay vì sử dụng trục tích hợp dịch vụ (ESB) cồng kềnh, hệ thống chuyển sang triết lý "Smart endpoints, Dumb pipes" của Microservices (sử dụng API Gateway và Kafka).

Kiến trúc phần mềm giống như việc quy hoạch một khuôn viên trường Đại học. Ban đầu, mọi thứ (phòng học, thư viện, giáo vụ) có thể nhồi nhét chung trong một tòa nhà đa năng (Monolith). Nhưng khi quy mô tăng lên, chúng ta phải xây tách rời thành khu giảng đường, thư viện, ký túc xá (Microservices), và cần quy hoạch hệ thống đường nội bộ, biển chỉ dẫn (API Gateway/Event Bus) để liên kết chúng.

Mỗi chương giải quyết một vấn đề cụ thể của KBM trong quá trình "quy hoạch" đó:

Bảng K.1: *Quá trình KBM qua 12 chương — vấn đề và giải pháp*

Ch.	Vấn đề KBM đối mặt	Giải pháp (bài học chương)
1	Ba monolith rời rạc: logic trùng lặp, danh tính không liên thông, chi phí bảo trì nhân lên	Quyết định chuyển đổi có cơ sở (Decision Framework)
2	Không rõ ranh giới service, entity-driven design	DDD + Event Storming → 7 Bounded Contexts core
3	API không nhất quán, breaking change thường xuyên	REST + OpenAPI + versioning strategy
4	Một dịch vụ lỗi gây hiệu ứng dây chuyền (cascading failure)	Circuit breaker + retry + timeout (Resilience4j)

Ch.	Vấn đề KBM đối mặt	Giải pháp (bài học chương)
5	Chế độ thi: hàng trăm bài nộp đồng thời, cần ordering + replay	Kafka event streaming + idempotent consumer
6	Submit flow stuck PENDING khi Judge crash, cần atomicity	Saga Choreography + compensation + semantic lock + timeout
7	Shared database Core ↔ Academic; leaderboard read-heavy	Database-per-service + CQRS cho leaderboard
8	Mỗi client, mỗi service tự kiểm tra danh tính — trùng lặp, dễ lệch chính sách	API Gateway tập trung: routing, xác minh JWT ở biên, thiết kế giới hạn lưu lượng
9	JWT HS256 shared secret, token tồn tại quá lâu	Custom Auth Service: HS256 → RS256 + refresh token rotation
10	Không thể Big Bang rewrite — hệ thống đang vận hành	Strangler Fig — tách dần từng dịch vụ, mang lại giá trị tức thì
11	Lỗi xảy ra nhưng không biết tại đâu, tại sao	Structured logging + distributed tracing + cảnh báo theo SLO
12	Deploy N service thủ công = rủi ro và chậm	Docker Compose + CI/CD tự động (GitLab CI) + rollback plan

Lộ trình chuyển đổi thực tế

Biết các pattern là điều kiện cần, nhưng **thứ tự áp dụng** mới là điều kiện đủ. Không phải tất cả pattern đều có giá trị như nhau ở từng giai đoạn. Nguyên tắc xuyên suốt: **Quick Wins trước, thay đổi rủi ro cao sau** — mỗi giai đoạn phải mang lại giá trị ngay, không cần đợi hoàn thành toàn bộ.

Lộ trình chi tiết cho KBM — bốn giai đoạn *Quick Wins → Observability Foundation → Resilience & CI/CD → Database Decomposition*, kèm bảng action theo Effort × Impact và các sai lầm thường gặp — đã được trình bày đầy đủ ở **Chương 10** (§10.5, *Lộ trình chuyển đổi — 4 giai đoạn*). Điểm mấu chốt của thứ tự này: phải *nhìn thấy* được hệ thống (observability) và đảm bảo khả năng chịu lỗi (resilience) *trước khi* tách database — bước rủi ro nhất.

Giá trị trước, pattern sau

> "Chưa cần implement CQRS chỉ vì nó hấp dẫn về mặt kỹ thuật. Hãy implement khi leaderboard query làm chậm database dùng chung của Core/Academic. Mọi pattern trong sách này đều được áp dụng để giải quyết vấn đề cụ thể — không phải để bản vẽ kiến trúc trông có vẻ hoàn chỉnh."

Khi nào áp dụng — heuristic thực tế

Không phải mọi hệ thống đều cần tất cả 12 chương. Bảng dưới đây tóm tắt các tín hiệu thực tế (heuristic) để ra quyết định:

Bảng K.2: *Heuristics — khi nào áp dụng pattern nào*

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Nhiều nhóm phải deploy chung và chờ nhau (Ch.1 §1.6, ma trận Ch.10)	Cân nhắc tách service theo bounded context	Ch.1, 10
Response time tăng khi có spike	Circuit breaker + async decoupling	Ch.4, 5

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Nhiều service cần cùng dữ liệu (JOIN xuyên service)	API Composition / CQRS	Ch.7
Giao dịch xuyên 2+ service	Saga + compensation	Ch.6
Không biết lỗi xảy ra ở đâu	Distributed tracing (`traceId`) + `requestId` / `correlationId`	Ch.11
Deploy mất >30 phút hoặc cần nhiều người	CI/CD automation	Ch.12
Token compromise nhưng không thể revoke	Short-lived JWT + refresh rotation	Ch.9

Điều không thay đổi

Công cụ thay đổi — Spring Boot hôm nay có thể khác 5 năm tới. Nhưng các nguyên tắc cốt lõi được học trong sách này không phụ thuộc vào framework cụ thể:

- **Kiến trúc hướng dịch vụ (SOA) là nền tảng**: dù công nghệ thay đổi (từ SOAP sang REST/gRPC, từ ESB sang Kafka), nhưng triết lý định nghĩa ranh giới dịch vụ và hợp đồng giao tiếp chuẩn hóa (Standardized Service Contract) vẫn luôn là bất biến.
- **Conway's Law**: cấu trúc hệ thống phản ánh cấu trúc tổ chức — thay đổi một cái phải đi kèm thay đổi kia.
- **Failure as first-class citizen**: microservices không loại bỏ failure, chúng làm failure trở thành tường minh. Thiết kế cho failure từ đầu.
- **Eventual consistency là trade-off, không phải khuyết điểm**: hiểu khi nào hệ thống cần strong consistency (điểm số, kết quả chấm bài) và khi nào có thể chấp nhận eventual (leaderboard).
- **Observability trước optimization**: không thể cải thiện những gì không đo được. Tracing và metrics là công cụ tư duy, không chỉ là công cụ debug.
- **Monolith First khi phù hợp**: bộ công cụ đã học không phải lúc nào cũng nên áp dụng — sự phán đoán về *khi nào* quan trọng hơn kỹ năng *làm thế nào*.

KBM sau lộ trình: một kịch bản

Nếu KBM đi đúng lộ trình ở Ch.10: sau các quick win và nền tảng observability (Giai đoạn 1–2), đội ngũ nhìn thấy lỗi nằm ở đâu thay vì đoán qua log từng container. Khi có resilience, dead letter topic và timeout cho saga nộp bài (Giai đoạn 3), không còn bài nộp nào treo ở PENDING. Đến khi tách schema và thay truy vấn chéo bằng API (Giai đoạn 4), Academic Service mới deploy được độc lập với Core. Trong kỳ thi, dashboard cho thấy độ trễ nộp bài nằm trong SLO của chế độ thi (Ch.11). Đây không phải kịch bản lý tưởng — đây là kết quả có thể đạt được khi áp dụng có trình tự và có phán đoán.

Quá trình học không kết thúc ở chương này. Lĩnh vực tiếp tục phát triển — service mesh, platform engineering, AI-assisted observability — nhưng mọi công nghệ mới đều xây dựng trên nền tảng kiến trúc hướng dịch vụ (SOA) và microservices đã học trong suốt cuốn sách này: bounded contexts, failure isolation, và deployment automation.